

Week-5 (DAY-23) Hands-On – Keerthivasan R

Level-1 Problem 1:

Scenario

A financial application needs to process multiple reports simultaneously to reduce waiting time. Instead of executing tasks sequentially, the system should run them concurrently using **C# Tasks** so that reports are generated faster.

Requirements

1. Create three methods:
 - a. GenerateSalesReport()
 - b. GenerateInventoryReport()
 - c. GenerateCustomerReport()
2. Each method should simulate processing time using Thread.Sleep() or Task.Delay().
3. Execute all three operations **concurrently using Task**.
4. Display a message when each report starts and when it finishes.
5. Display a final message once all reports are completed.

Technical Constraints

- Use **Task class from System.Threading.Tasks**.
- Use **Task.Run()** to execute methods.
- Use Task.WaitAll() or await **Task.WhenAll()** to wait for completion.
- The program must run in a **Console Application**.

Expectations

The program should start multiple report-generation tasks simultaneously and display completion messages for each report along with a final message once all tasks are completed.

Learning Outcome

Students will learn:

- How to create and run **Tasks in C#**
- How to execute **multiple operations concurrently**
- How to **wait for multiple tasks to complete**

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY_23
```

```

{
    internal class Problem_01
    {
        static void GenerateSalesReport()
        {
            Console.WriteLine("Sales report started...");
            Thread.Sleep(2000);
            Console.WriteLine("\nSales report completed");
        }

        static void GenerateInventoryReport()
        {
            Console.WriteLine("Inventory report started");
            Thread.Sleep(3000);
            Console.WriteLine("Inventory report completed");
        }

        static void GenerateCustomerReport()
        {
            Console.WriteLine("Customer report started");
            Thread.Sleep(2500);
            Console.WriteLine("Customer report completed");
        }

        static void Main(string[] args)
        {
            Task t1 = Task.Run(() => GenerateSalesReport());
            Task t2 = Task.Run(() => GenerateInventoryReport());
            Task t3 = Task.Run(() => GenerateCustomerReport());

            Task.WaitAll(t1, t2, t3);

            Console.WriteLine("\nAll reports are generated successfully!");
        }
    }
}

```

OUTPUT:

```

Inventory report started
Customer report started
Sales report started...

Sales report completed
Customer report completed
Inventory report completed

All reports are generated successfully!

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-23\HandsOnDAY-23\HandsOnDAY-23
\bin\Debug\net10.0-windows\HandsOnDAY-23.exe (process 2644) exited with cod
e 0 (0x0).
Press any key to close this window . . .|

```

Level-1 Problem 2: Asynchronous File Logger

Scenario:

An application writes logs to a file whenever an event occurs. Writing logs synchronously can slow down the application. Asynchronous file writing improves performance.

Requirements:

- Create an asynchronous method WriteLogAsync(string message).
- The method should simulate file writing using Task.Delay().
- Call this method multiple times to simulate logging different events.

Technical Constraints:

- Use async and await keywords.
- Use Task.Delay() to simulate file I/O.
- Use a console application.

Expectations:

- Logs should be written asynchronously.
- The main thread should remain responsive while logging operations occur.

Learning Outcome:

Students will learn how asynchronous operations improve performance when dealing with I/O operations.

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY_23
{
    internal class Problem_02
    {
        static async Task WriteLogAsync(string message)
        {
            Console.WriteLine("\nWriting Log: " + message);
            await Task.Delay(1000);
            Console.WriteLine("\nLog Written: " + message);
        }

        static async Task Main(string[] args)
```

```

    {
        Task t1 = WriteLogAsync("User logged in");
        Task t2 = WriteLogAsync("File uploaded");
        Task t3 = WriteLogAsync("Error occurred");

        await Task.WhenAll(t1, t2, t3);

        Console.WriteLine("\nAll logs written successfully");
    }
}

```

OUTPUT:

```

Writing Log: User logged in
Writing Log: File uploaded
Writing Log: Error occurred
Log Written: Error occurred
Log Written: File uploaded
Log Written: User logged in
All logs written successfully

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-23\HandsOnDAY-23\HandsOnDAY-23\bin\Debug
\net10.0-windows\HandsOnDAY-23.exe (process 28460) exited with code 0 (0x0).
Press any key to close this window . . .

```

Level-2 Problem 1: Asynchronous Order Processing System

Scenario:

An e-commerce system processes customer orders. Each order requires multiple steps such as payment verification, inventory check, and order confirmation. These steps involve delays and should be handled asynchronously.

Requirements:

- Create asynchronous methods:
 - VerifyPaymentAsync()
 - CheckInventoryAsync()
 - ConfirmOrderAsync()

- Simulate processing delays using Task.Delay().
- Execute steps asynchronously while maintaining the logical order of operations.

Technical Constraints:

- Use async and await.
- Use Task-based asynchronous methods.
- Implement using a console application.

Expectations:

- Each processing step should run asynchronously.
- The program should display messages indicating the progress of order processing.

Learning Outcome:

Students will understand how to design real-world workflows using asynchronous methods with async/await.

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY_23
{
    internal class Problem_03
    {
        static async Task VerifyPaymentAsync()
        {
            Console.WriteLine("Verifying Payment");
            await Task.Delay(2000);
            Console.WriteLine("Payment Verified");
        }

        static async Task CheckInventoryAsync()
        {
            Console.WriteLine("\nChecking Inventory");
            await Task.Delay(2000);
            Console.WriteLine("Inventory Available");
        }

        static async Task ConfirmOrderAsync()
        {
            Console.WriteLine("\nConfirming Order");
            await Task.Delay(2000);
            Console.WriteLine("Order Confirmed");
        }

        static async Task Main(string[] args)
        {

```

```

        await VerifyPaymentAsync();
        await CheckInventoryAsync();
        await ConfirmOrderAsync();

        Console.WriteLine("\nOrder Process Completed");
    }
}

```

OUTPUT:

```

Verifying Payment
Payment Verified

Checking Inventory
Inventory Available

Confirming Order
Order Confirmed

Order Process Completed

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-23\HandsOnDAY-23\HandsOnDAY-23\bin\Debug
\net10.0-windows\HandsOnDAY-23.exe (process 27168) exited with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 2: Debugging Incorrect Discount Calculation

Scenario

A retail application calculates the final price of products after applying a discount. Recently, users reported that the final price shown by the application is incorrect. The development team needs to **debug the application to identify where the incorrect calculation is happening**.

Requirements

- Create a console application that calculates the final product price.
- The program should accept:
 - Product Name
 - Product Price
 - Discount Percentage

- The final price should be calculated using the formula:

$$\text{FinalPrice} = \text{Price} - (\text{Price} \times \text{Discount} / 100)$$

- Use **debugging tools** to verify that the calculation is correct.
- Place **breakpoints** and inspect variable values during execution.

Technical Constraints

- Use **Visual Studio Debugging Tools**.
- Use **breakpoints, step over, step into, and watch window**.
- Implement the solution using a **.NET console application**.

Expectations

- Students should run the program in **debug mode**.
- They should track variable values and confirm that the discount calculation is correct.
- If incorrect results appear, students should identify the faulty logic.

Learning Outcome

Students will learn how to:

- Use **breakpoints effectively**.
- Inspect **variable values during program execution**.
- Identify logical errors using **debugging techniques**.

CODE:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace HandsOnDAY_23
{
    internal class Problem_04
    {
        static void Main(string[] args)
        {
            Console.Write("Enter Product Name: ");
            string name = Console.ReadLine();

            Console.Write("Enter Price: ");
            double price = Convert.ToDouble(Console.ReadLine());

            Console.Write("Enter Discount %: ");
            double discount = Convert.ToDouble(Console.ReadLine());
```

```

        // breakpoint
        double finalPrice = price - (price * discount / 100);

        Console.WriteLine("Final Price: " + finalPrice);
    }
}

```

OUTPUT:

```

Enter Product Name: Laptop
Enter Price: 10000
Enter Discount %: 10
Final Price: 9000

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-23\HandsOnDAY-23\HandsOnDAY-23\bin\Debug
\net10.0-windows\HandsOnDAY-23.exe (process 12184) exited with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 3: Application Tracing for Order Processing

Scenario

An e-commerce application processes customer orders. Sometimes the order processing fails, but developers are unable to identify where the failure occurs. The team decides to implement **Tracing** to monitor the execution flow and diagnose the issue.

Requirements

- Create a console application that simulates **order processing**.
- The order processing should include the following steps:
 1. Validate Order
 2. Process Payment
 3. Update Inventory

4. Generate Invoice

- Use **Trace class** to log messages at each step of the process.
- Display trace messages showing the execution flow.

Technical Constraints

- Use **System.Diagnostics.Trace**.
- Write trace messages using:
 - `Trace.WriteLine()`
 - `Trace.TraceInformation()`
- Configure a **TextWriterTraceListener** to store trace logs in a file.
- Implement the solution using **.NET console application**.

Expectations

- The application should log messages for each stage of order processing.
- Trace logs should help developers identify where failures occur.
- The trace output should be stored in a **log file**.

Learning Outcome

Students will learn how to:

- Implement **application tracing** using `System.Diagnostics`.
- Monitor application flow using **Trace listeners**.
- Use trace logs to **diagnose runtime issues in real-world applications**.

CODE:

```
using System;
using System.Diagnostics;
using System.IO;

namespace HandsOnDAY_23
{
    internal class Problem_05
    {
        static void Main(string[] args)
        {
            Trace.Listeners.Add(new
            TextWriterTraceListener(@"C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-23\traceLog.txt"));
            Trace.AutoFlush = true;

            Trace.WriteLine("Order Processing Started");
```

```

        ValidateOrder();
        ProcessPayment();
        UpdateInventory();
        GenerateInvoice();

        Trace.WriteLine("Order Processing Completed");

        Console.WriteLine("Tracing completed. Check traceLog.txt");
    }

    static void ValidateOrder()
    {
        Trace.TraceInformation("Validating Order...");
    }

    static void ProcessPayment()
    {
        Trace.TraceInformation("Processing Payment...");
    }

    static void UpdateInventory()
    {
        Trace.TraceInformation("Updating Inventory...");
    }

    static void GenerateInvoice()
    {
        Trace.TraceInformation("Generating Invoice...");
    }
}

```

OUTPUT:

```
Tracing completed. Check traceLog.txt
```

```

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-23\HandsOnDAY-23\HandsOnDAY-23\bin\Debug
\net10.0-windows\HandsOnDAY-23.exe (process 11112) exited with code 0 (0x0).
Press any key to close this window . . .|

```

```

Order Processing Started
HandsOnDAY-23 Information: 0 : Validating Order...
HandsOnDAY-23 Information: 0 : Processing Payment...
HandsOnDAY-23 Information: 0 : Updating Inventory...
HandsOnDAY-23 Information: 0 : Generating Invoice...
Order Processing Completed
|

```